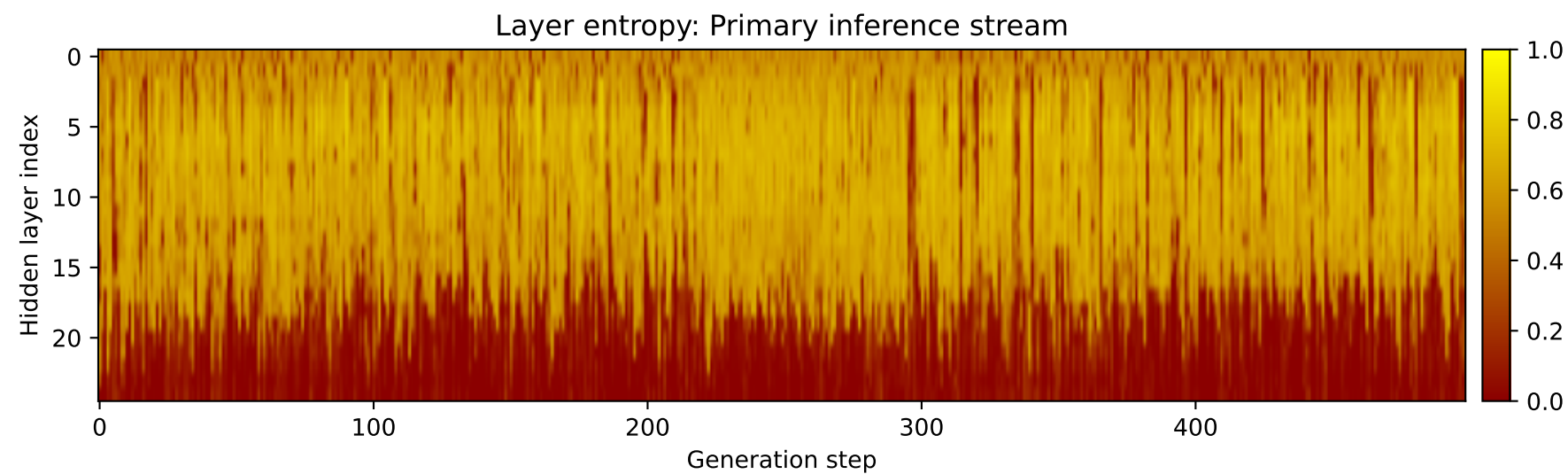
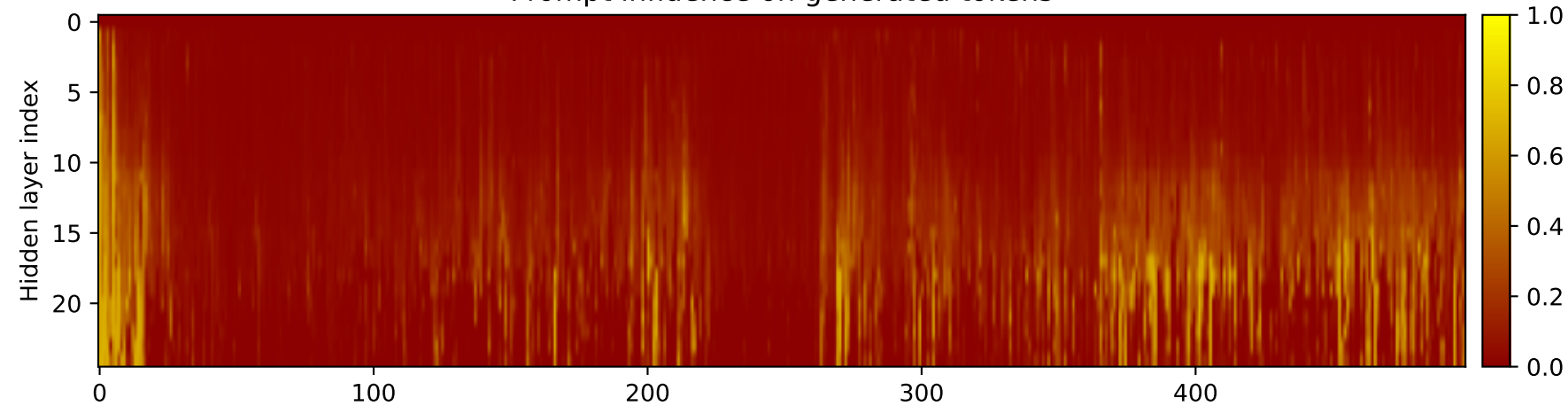
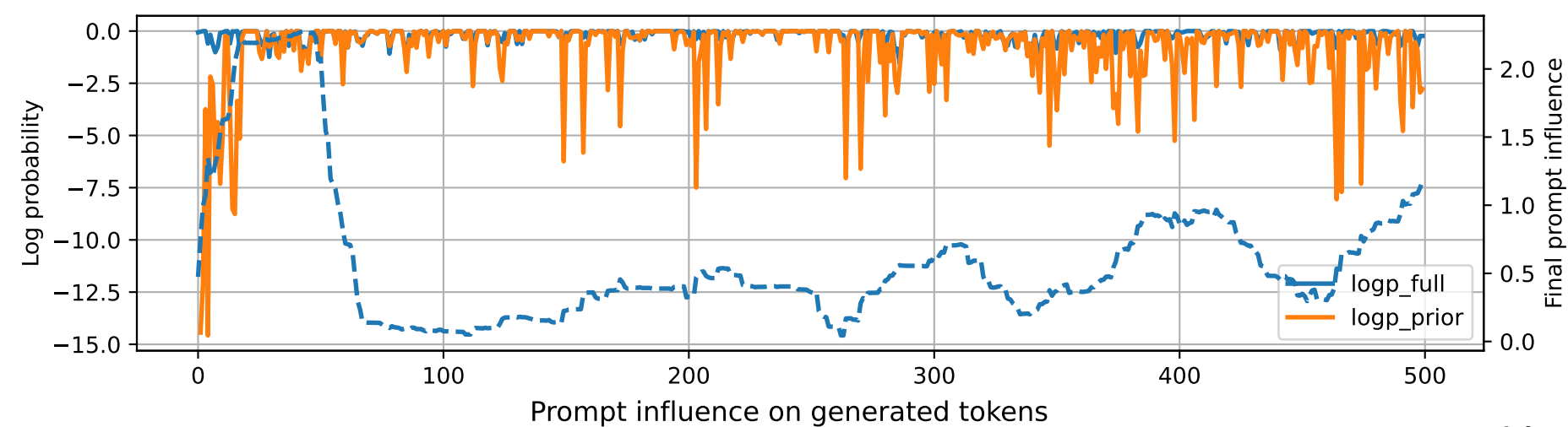


Prompt 1



PROMPT:
Task ID: PYC061 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC061 Your code must contain this required symbol:
def bfs Implement this task: define
function bfs(graph,start) returning visit order.. Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: BFS # Define function bfs(graph, start) returning
visit order. def bfs(graph, start): #
Create an empty queue to store the visited nodes queue = []
Initialize a visited list to keep track
of visited nodes visited = [] # Set the start node as
the first node to be visited
visited.append(start) # While the queue is not empty,
dequeue the first node from the queue while
queue: # Get the current node from the queue
current_node = queue.pop(0) # If the
current_node is not visited, add it to the visited list
if current_node not in visited:
visited.append(current_node) # For each neighbor of the
current node, enqueue it in the queue
for neighbor in graph[current_node]: # If the
neighbor is not visited, enqueue it in the queue
if neighbor not in visited:
queue.append(neighbor) # Return the visit order of the
visited nodes return visited # Example usage: graph = {
'A': ['B', 'C'], 'B': ['D', 'E'],
'C': ['F'], 'D': [], 'E': [], 'F': [] } start_node =
'A' visit_order = bfs(graph, start_node)
print(f"Visit order of nodes starting from {start_node}:
{visit_order}") ``` In this implementation, we
define a function `bfs` that takes a graph and a start node as
input. The graph is represented as a dictionary
where each node is a key and its neighbors are the values. The
function returns a list of visited nodes, which
represents the order in which nodes were visited during the
breadth-first search traversal. The function
starts by creating an empty queue `queue` to store the visited
nodes and initializing a visited list `visited`
to keep track of visited nodes. The start node is set as the
first node to be visited. The function then
enters a while loop that continues until the queue is empty. In
each iteration of the loop, the current node
is dequeued from the queue and added to the visited list. If the
current node is not visited, it is added to
the visited list. For each neighbor of the current node, the
function enqueues it in the queue. If the
neighbor is not visited, it is also enqueued in the queue. After
the loop